

Session vs. token auth

COMPLETE LEARNING GUIDE

What You'll Learn

Every authenticated backend has to answer the same basic question on every request: 'who is this, and are they still logged in?' — and there are two fundamentally different architectural answers to it. Session-based authentication keeps the actual login state on the server (in memory, a database, or a store like Redis) and gives the client only an opaque reference to it via a cookie. Token-based authentication (typically JWTs) encodes the login state directly into a signed token the client holds and presents on every request, with no server-side lookup required to validate it. This guide covers how each works end to end, the real security tradeoffs (particularly around XSS and token storage), and the practical rule of thumb for choosing between them based on the kind of client (server-rendered web app, SPA, mobile app, or microservices) being built.

1. Stateful Sessions: The Server Remembers, the Client Just Points

In session-based auth, the server creates a session record on successful login — an object holding the user's ID and any other login state — stores it server-side (in memory for a single small server, or in a shared store like Redis/a database for multiple servers), and sends the client only a random, unguessable session ID in a cookie. On every subsequent request, the client automatically sends that cookie back (cookies are attached by the browser automatically), and the server looks up the session by ID to find out who's making the request. Logging out is trivial and immediate: delete the session record server-side, and the session ID becomes meaningless even if an attacker still has the cookie.

JAVASCRIPT

```
const session = require("express-session");

app.use(session({
  secret: process.env.SESSION_SECRET,
  resave: false,
  saveUninitialized: false,
  cookie: { httpOnly: true, secure: true, maxAge: 24 * 60 * 60 * 1000 },
}));

app.post("/login", async (req, res) => {
  const user = await verifyCredentials(req.body.email, req.body.password);
  if (!user) return res.status(401).json({ error: "Invalid credentials" });
  req.session.userId = user.id; // session store now remembers this login
  res.status(200).json({ message: "Logged in" });
});
```

});

2. Stateless Tokens: JWT Structure and How Verification Works

A JSON Web Token (JWT) packs three base64url-encoded parts separated by dots — a header (algorithm/type), a payload (claims: user ID, role, expiration), and a signature — where the signature is a cryptographic hash of the header and payload, computed with a server-held secret. Anyone can decode and read a JWT's header and payload (it is not encryption, only encoding), but only someone holding the signing secret can produce a signature that will verify correctly, which is what makes the token tamper-evident: change one character of the payload and the signature no longer matches. Crucially, verifying a JWT requires no database lookup at all — the server just recomputes the signature and checks the expiration claim, which is what makes tokens 'stateless.'

JAVASCRIPT

```
const jwt = require("jsonwebtoken");

function login(req, res) {
  // after verifying credentials:
  const accessToken = jwt.sign(
    { sub: user.id, role: user.role },
    process.env.JWT_SECRET,
    { expiresIn: "15m" }
  );
  res.cookie("accessToken", accessToken, { httpOnly: true, secure: true,
  sameSite: "strict" });
  res.status(200).json({ message: "Logged in" });
}

function requireAuth(req, res, next) {
  const token = req.cookies.accessToken;
  if (!token) return res.status(401).json({ error: "Not authenticated" });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET); // no DB lookup needed
    next();
  } catch {
    return res.status(401).json({ error: "Invalid or expired token" });
  }
}
```

3. Where to Store a Token on the Client: The XSS Problem

The single most consequential decision in token-based auth is where the client stores the token, and it comes down to a tradeoff between two different attack vectors. Storing a JWT in localStorage is convenient (readable by JavaScript) but exposes it directly to any successful XSS (cross-site scripting) attack — malicious script injected into the page can simply read localStorage and exfiltrate the token. Storing it in an httpOnly cookie makes it invisible to JavaScript entirely (immune to that specific XSS theft vector) but requires CSRF (cross-site request forgery) protections instead, since the browser will attach the cookie automatically to any request to that domain, including ones triggered by a malicious third-party page.

- localStorage: readable by any JavaScript on the page, including injected XSS payloads — high risk if the app has any XSS vulnerability.
- httpOnly cookie: invisible to JavaScript, immune to token theft via XSS, but needs CSRF protection (SameSite attribute, CSRF tokens).
- The current best-practice consensus for browser-based apps favors httpOnly, Secure, SameSite cookies over localStorage for holding tokens.
- Mobile and server-to-server clients don't have this browser-specific tradeoff and commonly store tokens in secure OS-level storage instead.

4. Refresh Tokens: Balancing Short Expiry With Usability

A JWT that never expires is a permanent credential an attacker who steals it can use forever, but a JWT with a very short expiry (minutes) forces constant re-logins, which is a poor user experience. The standard solution is a two-token pattern: a short-lived access token (minutes, sent with every request) and a long-lived refresh token (days/weeks, stored more restrictively and used only to request a new access token when the old one expires). This limits the window an attacker can exploit a stolen access token while still keeping the user logged in for a reasonable period, and revoking the refresh token server-side (which does require a database check, unlike access tokens) effectively logs the user out everywhere.

5. Revocation: The Core Limitation of Pure Token Auth

Because a stateless JWT is validated purely by cryptographic signature and an expiration timestamp, there is no built-in way to invalidate one before it naturally expires — the server doesn't track which tokens are 'still valid' anywhere. This is a real limitation compared to sessions, where logout means an immediate, guaranteed server-side deletion. Systems that need real revocation (an admin forcibly logging out a compromised account immediately) either keep access token lifetimes very short and revoke only the refresh token, or maintain a server-side denylist of revoked token IDs to check on each request — which reintroduces the server-side state pure JWTs were meant to avoid.

6. Choosing Between Them by Client Type

Session-based auth remains an excellent default for traditional server-rendered web apps, where the browser and server share cookies naturally and instant, guaranteed logout matters. Token-based auth is generally preferred for single-page apps calling a separate API domain, mobile apps (which have no browser cookie jar to rely on), and service-to-service/microservice communication, where a stateless token that any service can verify independently (without a shared session store) fits the architecture better. Many real systems use both: sessions or cookies for the primary web app, and short-lived tokens for a public API or mobile client consuming the same backend.

Common Pitfalls

- Storing a JWT in localStorage, where any XSS vulnerability anywhere in the app can exfiltrate it — httpOnly cookies are not vulnerable to this specific attack.
- Putting sensitive data (passwords, full PII) in a JWT payload, forgetting that a JWT is only encoded (base64), not encrypted, and readable by anyone who has it.
- Issuing access tokens with no expiration, or a needlessly long one, removing the main safety net that limits a stolen token's usefulness.
- Believing 'logging out' invalidates a JWT — a stateless JWT remains valid until it expires unless the server maintains an explicit denylist.
- Mixing session and token auth inconsistently across different parts of the same application without a clear reason, doubling the attack surface to reason about.
- Not setting the Secure and SameSite attributes on auth cookies, leaving them exposed over plain HTTP or vulnerable to cross-site request forgery.

Key Takeaways

- Sessions keep login state on the server and give the client only an opaque cookie ID — logout is immediate and server-enforced.
- JWTs encode login state directly into a signed, client-held token — verification needs no database lookup, but revocation before expiry is hard.
- A JWT's payload is readable by anyone (base64-encoded, not encrypted) — never put secrets in it, only non-sensitive claims.
- Prefer httpOnly, Secure, SameSite cookies over localStorage for storing tokens in browser-based apps, to avoid XSS token theft.
- The short-lived access token + long-lived refresh token pattern balances security (short exposure window) with usability (infrequent re-login).
- Choose sessions for traditional server-rendered apps and tokens for SPAs, mobile apps, and service-to-service communication — or use both where each fits.

Learn More

OWASP — JSON Web Token Cheat Sheet

https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html

jwt.io — Introduction to JSON Web Tokens

<https://jwt.io/introduction>

MDN — Using HTTP Cookies

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies>